Heterogeneous Active Retrieval Across Structurally Different Evidence Substrates

Working Draft

August 2026

Abstract

Adaptive generation-time retrieval is established prior art. We test a narrower systems question: do structurally different information needs benefit from distinct read-only evidence coprocessors rather than one universal text retriever? We implement typed relational, lexical, semantic-vector, and freshness-aware controlled-web adapters behind a credential-isolating registry. On a 22-item source-optimal controlled benchmark, source-native oracle and transparent heuristic routing both reach 22/22, including safe abstention on a conflicting fresh source. A matched universal textual index reaches 13/22 and supports 7/16 retrieval-required answers; it loses relational aggregate/join and source-status semantics. Broadcast also reaches 22/22 but averages 2.91 calls per example versus 0.73 for routed retrieval. Explicit descriptor burden grows from 4.0 to 22.5 mean tokens as the available catalog grows from one to four sources, while the runtime registry adds no recurring descriptor tokens. Replacing the controlled relational, lexical, and vector implementations with DuckDB, SQLite FTS5, and exact-flat FAISS preserves all 154 matched four-source decisions and complete required provenance for all three substituted sources. These remain small controlled runtime results, not language-model or production-data evidence. The pre-registered learned-router gate is NO-GO: native heterogeneity has value, context burden grows, and broadcast costs more, but the heuristic leaves no routing gap.

1 Question and Claim Boundary

FLARE and Self-RAG establish important forms of adaptive retrieval [1, 2]. Our question is whether source-native semantics matter after retrieval need has been identified. The core paper therefore keeps need and source routing transparent. It does not claim a learned router, production web-search quality, or neural answer use.

The factorized pipeline is

$$\text{NEED} \rightarrow \text{SOURCE} \rightarrow \text{QUERY} \rightarrow \text{RETRIEVE} \rightarrow \text{EVIDENCE STATUS} \rightarrow \text{USE}.$$

Paper 1.5 can provide a typed retrieval-required event. Paper 2.5 studies the source and query stages without importing a learned multi-source policy.

2 Read-Only Source Runtime

Every request contains a request identifier, source type, operation, typed payload, and bounds. Every evidence record contains source type and version, record identifier, value/content, observation time, relevance, support status, provenance, latency, and retrieved bytes. The runtime accepts only sources whose side-effect class is read-only. Credential scopes remain in the registry and are removed from public catalog descriptors.

2.1 Source-native adapters

- **Relational DB:** an embedded SQLite database with parameterized, typed templates for lookup, sum, count, average, argmax, and join.
- **Lexical index:** exact document/phrase retrieval over versioned policy and handbook records.
- **Vector index:** deterministic semantic-vector matching over reports, including controlled synonym normalization.
- **Fresh web analogue:** dated public-feed records with one explicit two-record conflict. It exercises freshness and conflict policy without network variability.

These are genuinely different access semantics. Multiple wrappers over one text index would not satisfy the design.

The backend interface is independently swappable. A local production-shaped suite substitutes DuckDB parameterized SQL, SQLite FTS5 ranking, and FAISS `IndexFlatIP` retrieval while retaining the same typed requests and evidence records. Each substituted record adds backend/version, resource, normalized query, record IDs, and snapshot to provenance. The web source remains the controlled analogue in this pass.

3 Routing and Typed Queries

The heuristic routes structured attributes and aggregates to DB, exact documents/clauses to lexical retrieval, paraphrased report questions to vector retrieval, and current/public requests to the fresh source. After source selection, source-specific fields compile to bounded requests such as `db.max_sales(year=2026)` or `web.lookup(entity,relation,time_window)`. Generated SQL and arbitrary code execution are forbidden.

4 Benchmark and Conditions

The frozen benchmark contains six DB tasks, three lexical tasks, three vector tasks, four web tasks, and six supplied-context controls. DB tasks cover every implemented operation. The web set contains a conflict whose authorized answer is abstention. Nested catalogs expose 1, 2, 3, and 4 source types.

For every catalog we run seven conditions:

1. oracle need, source, and query;
2. real typed need with oracle source;
3. real typed need with heuristic source;
4. heuristic need and source;
5. explicit source selection with all descriptors;
6. one universal textual retriever;
7. broadcast to every available native source.

Table 1: Four-source catalog over 22 examples. Support and UCR use the 16 retrieval-required tasks. Calls and fanout average over all examples.

Condition	Final	Support	UCR	Calls	Descriptor tok.
Native oracle	1.000	1.000	0.000	0.73	0.0
Heuristic routing	1.000	1.000	0.000	0.73	0.0
Explicit schemas	1.000	1.000	0.000	0.73	22.5
Universal text	0.591	0.438	0.562	0.73	3.6
Broadcast	1.000	1.000	0.000	2.91	0.0

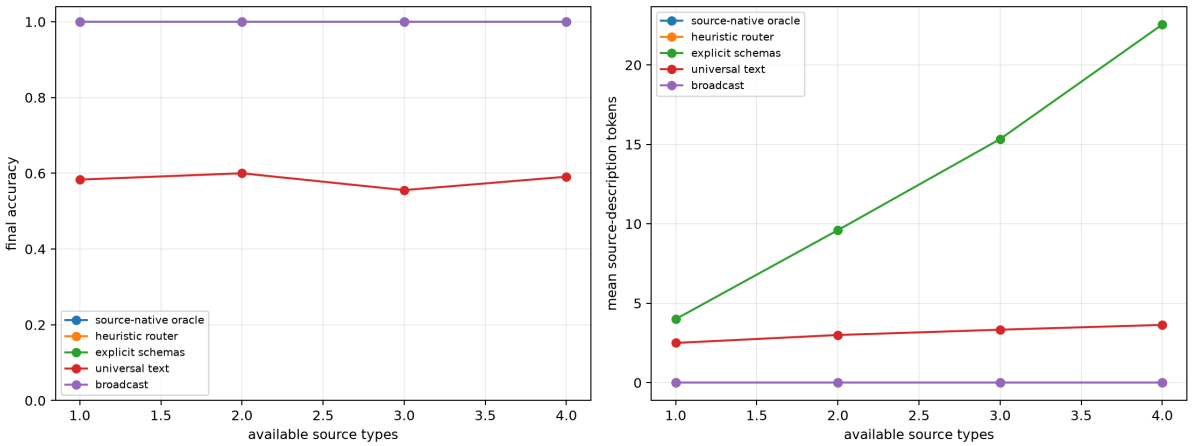

Figure 1: Final accuracy and recurring source-description tokens as source count grows. Registry capabilities remain outside neural context; explicit schemas grow with the catalog.

This yields 469 predictions and matching factorized traces. The universal index contains textual snapshots of every available source record and returns the top lexical match. It is intentionally strong on direct textual facts, but it cannot execute relational aggregates or preserve native conflict semantics.

5 Controlled Results

Native execution and routing are perfect by construction on this small source-optimal diagnostic, so the result is not a claim of broad routing accuracy. The useful comparison is semantic headroom: universal textualization misses nine examples that native source operations resolve. Broadcasting removes routing risk but performs four calls per retrieval-required example. The routed conditions call exactly one source on each of 16 needs and none on six controls.

5.1 Conflict and composition

The fresh-source conflict returns two provenance-bearing CONFLICT records. Native conditions accept abstention rather than selecting a value. Twenty-four additional deterministic compositions test entity resolver → DB and date resolver → web. Both stages and final outputs are 24/24, and every request records a dependency DAG. These are plumbing/economics checks, not model planning evidence.

Table 2: Local backend substitution on oracle-source rows. Latency is mean source execution time on one Windows host, not a backend ranking.

Backend	Rows	Final	Provenance	Mean ms
DuckDB	6	1.000	1.000	0.742
SQLite FTS5	3	1.000	1.000	0.036
FAISS flat	3	1.000	1.000	0.047

5.2 Local production-backend substitution

We rerun the complete four-source matrix after substituting DuckDB 1.5.5 for the embedded relational implementation, SQLite 3.49.1 FTS5 for lexical scanning, and FAISS 1.15.0 exact-flat inner-product search for the controlled vector search. All 154 matched predictions retain the same final and evidence-support decisions. Table 2 isolates oracle-source rows for the three replaced backends. Required provenance completeness checks backend/version, resource, normalized query, record IDs, and snapshot.

This validates adapter substitution and provenance retention on the frozen data; it does not test service deployment, persistence, concurrency, failure recovery, or realistic corpus scale. No Docker service was started. Postgres/pgvector, Iceberg catalogs, Qdrant, Weaviate, semantic layers, and ontologies are outside this completed local phase.

For the next boundary, the repository includes a WSL2-only Compose definition, seed SQL, health check, explicit Postgres and pgvector adapters, and integration tests gated by a caller-supplied DSN. Both adapters use parameter binding and read-only transactions; unavailable services skip rather than fall back. These assets were statically checked only. They contribute no result or latency claim.

6 Paper 3.5 Gate

Learned source routing requires all four pre-registered criteria:

1. source-native oracle value over universal retrieval: **pass**;
2. a measurable heuristic-to-oracle gap: **fail**;
3. source descriptor burden grows with catalog size: **pass**;
4. broadcast does not dominate quality and cost: **pass**.

The decision is therefore NO-GO. The negative gate is substantive: a learned router would add training and failure surface without measured routing headroom on this benchmark. A later, independently frozen natural-language set may reopen the gate; this result does not.

7 Limitations and Falsification

The corpus is synthetic, tiny, single-seed, and designed so source semantics are clear. The heuristic sees controlled lexical cues and therefore should not be generalized to natural requests. FAISS indexes the same controlled semantic features rather than a production embedding model. The web adapter is local and cannot measure network failures, live ranking, or monetary cost. Final accuracy is deterministic evidence acceptance, not LLM evidence use. Backend substitution does

not establish service reliability or enterprise-data validity, and all latencies are local prototype measurements.

Heterogeneous routing would be weakened if a stronger matched universal system executes aggregates, preserves status/provenance, and closes the 9/22 gap at comparable cost; if broadcast is cheap enough to dominate; or if natural source-selection errors erase native source value.

8 Reproduction

Code is under `src/ccpu/paper2_5`; reusable evidence contracts are under `src/ccpu/common`. From the repository root:

```
python -m ccpu paper2.5 freeze \
  --output-dir artifacts/paper2_5/next_iter/data_v2
python -m ccpu paper2.5 run \
  --benchmark artifacts/paper2_5/next_iter/data_v2/benchmark.jsonl \
  --source-count 4 \
  --output-dir artifacts/paper2_5/next_iter/source_n4_v2
python -m ccpu paper2.5 analyze --predictions <n1> <n2> <n3> <n4> \
  --output-dir artifacts/paper2_5/next_iter/analysis_v2
python -m ccpu paper2.5 compositions --count-per-family 12 \
  --output-dir artifacts/paper2_5/next_iter/compositions_v1
python -m ccpu paper2.5 run \
  --benchmark artifacts/paper2_5/production_v1/data/benchmark.jsonl \
  --source-count 4 --backend-suite local_production \
  --output-dir artifacts/paper2_5/production_v1/source_n4
python -m ccpu paper2.5 analyze-production \
  --controlled-predictions <controlled-n4-predictions> \
  --production-predictions <production-n4-predictions> \
  --production-traces <production-n4-traces> \
  --output-dir artifacts/paper2_5/production_v1/substitution
```

Run manifests hash benchmark, predictions, traces, and summaries and record the repository and environment state. Install the `data` project extra for DuckDB and FAISS. This matrix uses no network, model-token, or container service.

9 Conclusion

Source-native execution has clear controlled value over universal textualization, and routing avoids broadcast fanout while keeping capability descriptors outside recurring context. That result justifies heterogeneous runtime adapters, not a learned router. Because the transparent heuristic closes the oracle routing gap on the present freeze, Paper 3.5 remains gated off. The local substitution result strengthens the interface claim: source-native semantics and provenance survive three real in-process backend replacements. It does not yet establish the broader production-data-stack claim.

References

[1] Z. Jiang et al. Active retrieval augmented generation. *EMNLP*, 2023. `https://arxiv.org/abs/2305.06983`.

[2] A. Asai et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024. https://arxiv.org/abs/2310.11511.